

Technical video transcript

Slide 0 — Team Introduction

Welcome to SmartResearch, a capstone project supervised by Dr. Jack Yang.

Our team is Chelsea, Nishad, and Noor, and we developed SmartResearch as an AI-powered platform for managing academic research papers.

The system helps users upload papers, extract key information, generate summaries, search by meaning, group related research, and create structured reports.

This presentation will show how SmartResearch works, how it was built, and how it supports a more efficient research workflow.

Slide 1 — Project Overview

SmartResearch was developed to reduce the manual effort involved in managing academic literature.

Researchers often work with large collections of PDF papers, which can be difficult to organise, search, summarise, and compare manually. This can slow down literature reviews and make it harder to identify relevant research.

SmartResearch brings these tasks into one platform. Users can upload papers, extract text and metadata, generate summaries, search by meaning rather than exact keywords, group related papers through clustering, and generate structured reports.

The aim is to support faster and more organised research workflows by combining document management with AI-assisted analysis.

Slide 2 — System Architecture

SmartResearch uses a layered architecture with separate frontend, backend, AI processing, and storage components.

The frontend is built with React and Vite, and provides the interface for uploading papers, viewing documents, searching content, and generating reports. The backend uses FastAPI to handle API requests, manage the document processing workflow, and connect the interface to the AI and NLP services.

The AI processing layer supports text extraction, summarisation, metadata processing, semantic search, and clustering, using tools such as PyPDF2, Tesseract OCR, SPECTER2 embeddings, and KMeans clustering. The storage layer keeps uploaded documents and

processed outputs, including summaries, metadata, embeddings, clustering results, and reports.

This separation gives each part of the system a clear responsibility, making the project easier to test, maintain, and extend.

Slide 3 — Frontend Overview

The SmartResearch frontend was designed with usability and simplicity in mind.

The application follows a modular page-based architecture where each major feature is implemented as an independent page and component.

Reusable components help reduce duplication and improve maintainability across the application.

The responsive design ensures a consistent user experience while keeping navigation simple and intuitive.

The frontend acts as the interface between the user and the backend, providing easy access to all major research management features.

Slide 4 — Backend Overview

The backend serves as the core processing engine of SmartResearch.

It is responsible for handling requests, managing document processing workflows, coordinating NLP services, and returning structured responses to the frontend.

The system uses a modular architecture where different processing tasks are separated into independent services. This approach improves maintainability and allows new features to be added more easily in the future.

Communication between the frontend and backend occurs through REST APIs, with all responses returned in JSON format.

This modular design allows SmartResearch to efficiently manage multiple processing pipelines while keeping the system organized and scalable.

Slide 5 — Storage Layer

The storage layer plays an important role in managing all processed research information.

When a document is uploaded, the system stores not only the original PDF but also the extracted text, metadata, generated summaries, and semantic embeddings.

Each document is assigned a unique identifier, allowing the system to efficiently organize and retrieve information.

This structured storage approach supports semantic search, clustering, report generation, and document management while maintaining consistent access to research data.

Slide 6 — Upload Page

The Upload Page serves as the main entry point for new research documents.

Users can upload one or multiple PDF files through a simple drag-and-drop interface.

The system validates uploaded files and provides real-time processing feedback to the user.

Once uploaded, the document is transferred to the backend, where it enters the document processing pipeline.

This interface was designed to make the upload process simple and accessible while supporting large collections of research papers.

Slide 7 — Document Processing Pipeline

After a document is uploaded, SmartResearch automatically processes it through several stages.

The first step is text extraction using PyPDF2, which extracts text from standard PDF documents.

However, many academic papers are available as scanned images rather than selectable text. In these cases, the system automatically uses Tesseract OCR as a fallback mechanism to extract readable content.

Once the text is available, metadata extraction identifies information such as titles, authors, publication years, and other bibliographic details.

The document is then passed through the summarization pipeline, which generates concise summaries that help users quickly understand the content of a paper.

Finally, all processed information is stored for future search, clustering, and report generation.

Slide 8 — All Papers Dashboard

The All Papers Dashboard acts as a centralized repository for uploaded research papers.

Each document is displayed together with its metadata, generated summary, publication details, and relevance information.

This allows users to browse large collections of research papers without needing to open every document individually.

The dashboard provides a structured and organized view of research information, making literature management more efficient and accessible.

One of the most important features of SmartResearch is its semantic search capability.

Traditional keyword search depends on exact word matches, which can sometimes miss relevant research papers that use different terminology.

Semantic search addresses this limitation by focusing on meaning rather than exact keywords.

The system uses SPECTER2 embeddings to convert research papers into vector representations.

When a user submits a query, SmartResearch compares the semantic meaning of the query against stored document embeddings and retrieves the most relevant papers.

Hybrid search combines keyword matching with semantic similarity, providing more accurate and meaningful search results while also supporting research recommendations.

Slide 9 — Research Clustering

The clustering module helps researchers organize large collections of papers automatically.

Instead of manually categorizing documents, SmartResearch groups papers based on semantic similarity.

The system generates SPECTER2 embeddings and applies KMeans clustering to identify groups of related research papers.

Keywords are extracted from each cluster to help users understand the main research themes represented in that group.

This functionality supports literature reviews, topic discovery, and research exploration by revealing relationships between papers that may not be immediately obvious.

Slide 10 — Export & Report Generation

The export module allows users to transform processed research information into structured reports.

Researchers often need summaries, metadata, and organized findings for academic review or presentation purposes.

SmartResearch automates this process by generating exportable research reports that combine information collected throughout the system.

This reduces manual documentation effort and helps researchers create organized outputs more efficiently.

The export functionality also supports sharing research findings with supervisors, collaborators, and academic teams.

Slide 11 — Challenges & Solutions

Throughout development, several technical challenges were encountered.

One major challenge involved scanned PDF documents that did not contain selectable text. This was addressed through OCR integration using Tesseract.

Another challenge involved processing very large research papers. To overcome this, chunk-based summarization techniques were introduced to improve processing reliability.

Metadata quality was also inconsistent across documents, leading to the use of metadata enrichment methods to improve accuracy.

Traditional keyword search often failed to identify semantically related papers, which motivated the implementation of semantic search using vector embeddings.

Finally, organizing large collections of research papers became increasingly difficult, leading to the development of clustering functionality for automated topic organization.

Together, these solutions significantly improved the accuracy, usability, and effectiveness of the system.

Slide 12 — Future Work

For future development, SmartResearch could be improved by integrating stronger AI models for higher-quality summaries and deeper document analysis.

Semantic search could also be improved with better embedding models and ranking strategies, so users receive more accurate results when searching by meaning rather than exact keywords.

Other future improvements include topic modelling, cloud deployment, multi-user collaboration, and analytics features. These would make the platform more practical for research teams and provide clearer insights into research trends, paper relationships, and gaps in a collection.

Slide 13 — Conclusion

In conclusion, SmartResearch provides a centralised platform for managing academic research workflows.

The system supports paper upload, text and metadata extraction, automatic summaries, semantic search, clustering, and report generation.

By combining document management with AI-powered analysis, SmartResearch reduces manual effort and helps users work with academic literature more efficiently.

Thank you for your time and attention. We hope you enjoyed our presentation